

Assembler Design

Assembler

- A system software program that translates assembly language source code into machine language object code.
- It translates mnemonic opcode to their machine language equivalent.
- It assigns machine addresses to symbolic label used by programmer.
- It performs:
 - Mnemonic → opcode conversion
 - Symbolic address → numeric address binding
 - Constant generation (BYTE, WORD)
 - Storage reservation (RESB, RESW)
 - Output of object program (Header, Text, End records)

Example:

LDA FIVE

ADD THREE

STA RESULT

The computer's CPU only understands **binary**

It also replaces symbolic names like **FIVE** with actual memory addresses.

Basic Assembler Functions

In addition to the mnemonic machine instructions, we have used the following assembler directives:

START: Specify name and starting address for the program.

END: Indicate the end of the source program and (optionally) specify the first executable instruction in the program.

BYTE: Generate character or hexadecimal constant, occupying as many bytes as needed to represent the constant.

WORD: Generate one-word integer constant.

RESB: Reserve the indicated number of bytes for a data area.

RESW: Reserve the indicated number of words for a data area.

The program contains a main routine that reads records from an input device (identified with device code F1) and copies them to an output device (code 05).

Line	Source statement			
5	COPY	START	1000	COPY FILE FROM INPUT TO OUTPUT
10	FIRST	STL	RETADR	SAVE RETURN ADDRESS
15	CLOOP	JSUB	RDREC	READ INPUT RECORD
20		LDA	LENGTH	TEST FOR EOF (LENGTH = 0)
25		COMP	ZERO	
30		JEQ	ENDFIL	EXIT IF EOF FOUND
35		JSUB	WRREC	WRITE OUTPUT RECORD
40		J	CLOOP	LOOP
45	ENDFIL	LDA	EOF	INSERT END OF FILE MARKER
50		STA	BUFFER	
55		LDA	THREE	SET LENGTH = 3
60		STA	LENGTH	
65		JSUB	WRREC	WRITE EOF
70		LDL	RETADR	GET RETURN ADDRESS
75		RSUB		RETURN TO CALLER
80	EOF	BYTE	C'EOF'	
85	THREE	WORD	3	
90	ZERO	WORD	0	
95	RETADR	RESW	1	
100	LENGTH	RESW	1	LENGTH OF RECORD
105	BUFFER	RESB	4096	4096-BYTE BUFFER AREA
110	.			
115	.	SUBROUTINE TO READ RECORD INTO BUFFER		
120	.			
125	RDREC	LDX	ZERO	CLEAR LOOP COUNTER
130		LDA	ZERO	CLEAR A TO ZERO
135	RLOOP	TD	INPUT	TEST INPUT DEVICE
140		JEQ	RLOOP	LOOP UNTIL READY
145		RD	INPUT	READ CHARACTER INTO REGISTER A
150		COMP	ZERO	TEST FOR END OF RECORD (X'00')
155		JEQ	EXIT	EXIT LOOP IF EOR
160		STCH	BUFFER,X	STORE CHARACTER IN BUFFER
165		TIX	MAXLEN	LOOP UNLESS MAX LENGTH
170		JLT	RLOOP	HAS BEEN REACHED
175	EXIT	STX	LENGTH	SAVE RECORD LENGTH
180		RSUB		RETURN TO CALLER
185	INPUT	BYTE	X'F1'	CODE FOR INPUT DEVICE
190	MAXLEN	WORD	4096	
195	.			
200	.	SUBROUTINE TO WRITE RECORD FROM BUFFER		
205	.			
210	WRREC	LDX	ZERO	CLEAR LOOP COUNTER
215	WLOOP	TD	OUTPUT	TEST OUTPUT DEVICE
220		JEQ	WLOOP	LOOP UNTIL READY
225		LDCH	BUFFER,X	GET CHARACTER FROM BUFFER
230		WD	OUTPUT	WRITE CHARACTER
235		TIX	LENGTH	LOOP UNTIL ALL CHARACTERS
240		JLT	WLOOP	HAVE BEEN WRITTEN
245		RSUB		RETURN TO CALLER
250	OUTPUT	BYTE	X'05'	CODE FOR OUTPUT DEVICE
255		END	FIRST	

Figure 2.1 Example of a SIC assembler language program.

Simple SIC Assembler

The translation of source program to object code requires us to accomplish the following functions (not necessarily in the order given):

1. Convert mnemonic operation codes to their machine language equivalents—e.g., translate STL to 14 (line 10).
2. Convert symbolic operands to their equivalent machine addresses— eg. translate RETADR to 1033 (line 10).
3. Build the machine instructions in the proper format.
4. Convert the data constants specified in the source program into their internal machine representations—e.g., translate EOF to 454F46 (line 80).
5. Write the object program and the assembly listing.

Line	Loc	Source statement			Object code
5	1000	COPY	START	1000	
10	1000	FIRST	STL	RETADR	141033
15	1003	CLOOP	JSUB	RDREC	482039
20	1006		LDA	LENGTH	001036
25	1009		COMP	ZERO	281030
30	100C		JEQ	ENDFIL	301015
35	100F		JSUB	WRREC	482061
40	1012		J	CLOOP	3C1003
45	1015	ENDFIL	LDA	EOF	00102A
50	1018		STA	BUFFER	0C1039
55	101B		LDA	THREE	00102D
60	101E		STA	LENGTH	0C1036
65	1021		JSUB	WRREC	482061
70	1024		LDL	RETADR	081033
75	1027		RSUB		4C0000
80	102A	EOF	BYTE	C'EOF'	454F46
85	102D	THREE	WORD	3	000003
90	1030	ZERO	WORD	0	000000
95	1033	RETADR	RESW	1	
100	1036	LENGTH	RESW	1	
105	1039	BUFFER	RESB	4096	
110		.			
115		.	SUBROUTINE TO READ RECORD INTO BUFFER		
120		.			
125	2039	RDREC	LDX	ZERO	041030
130	203C		LDA	ZERO	001030
135	203F	RLOOP	TD	INPUT	E0205D
140	2042		JEQ	RLOOP	30203F
145	2045		RD	INPUT	D8205D
150	2048		COMP	ZERO	281030
155	204B		JEQ	EXIT	302057
160	204E		STCH	BUFFER,X	549039
165	2051		TIK	MAXLEN	2C205E
170	2054		JLT	RLOOP	38203F
175	2057	EXIT	STX	LENGTH	101036
180	205A		RSUB		4C0000
185	205D	INPUT	BYTE	X'F1'	F1
190	205E	MAXLEN	WORD	4096	001000
195		.			
200		.	SUBROUTINE TO WRITE RECORD FROM BUFFER		
205		.			
210	2061	WRREC	LDX	ZERO	041030
215	2064	WLOOP	TD	OUTPUT	E02079
220	2067		JEQ	WLOOP	302064
225	206A		LDCH	BUFFER,X	509039
230	206D		WD	OUTPUT	DC2079
235	2070		TIK	LENGTH	2C1036
240	2073		JLT	WLOOP	382064
245	2076		RSUB		4C0000
250	2079	OUTPUT	BYTE	X'05'	05
255			END	FIRST	

Figure 2.2 Program from Fig. 2.1 with object code.

- This instruction contains a forward reference—that is, a reference to a label (RETADR) that is defined later in the program. If we attempt to translate the program line by line, we will be unable to process this statement because we do not know the address that will be assigned to RETADR.
- Because of this, most assemblers make two passes over the source program.
- The first pass does little more than scan the source program for label definitions and assign addresses (such as those in the Loc column in Fig. 2.2).
- The second pass performs most of the actual translation previously described.
- The assembler must write the generated object code onto some output device. This object program will later be loaded into memory for execution.
- The simple object program format we use contains three types of records:

Header, Text, and End.

- The **Header** record contains the program name, starting address, and length.
- **Text** records contain the translated (i.e., machine code) instructions and data of the program, together with an indication of the addresses where these are to be loaded.
- The **End** record marks the end of the object program and specifies the address in the program where execution is to begin.

Header record:

Col. 1	H
Col. 2-7	Program name
Col. 8-13	Starting address of object program (hexadecimal)
Col. 14-19	Length of object program in bytes (hexadecimal)

Text record:

Col. 1	T
Col. 2-7	Starting address for object code in this record (hexadecimal)
Col. 8-9	Length of object code in this record in bytes (hexadecimal)
Col. 10-69	Object code, represented in hexadecimal (2 columns per byte of object code)

End record:

Col. 1	E
Col. 2-7	Address of first executable instruction in object program (hexadecimal)

Two Pass Assembler

Pass 1 (define symbols):

1. Assign addresses to all statements in the program.
2. Save the values (addresses) assigned to all labels for use in Pass 2.
3. Perform some processing of assembler directives. (This includes processing that affects address assignment, such as determining the length of data areas defined by BYTE, RESW, etc.)

Pass 2 (assemble instructions and generate object program):

1. Assemble instructions (translating operation codes and looking up addresses).
2. Generate data values defined by BYTE, WORD, etc.
3. Perform processing of assembler directives not done during Pass 1.
4. Write the object program and the assembly listing

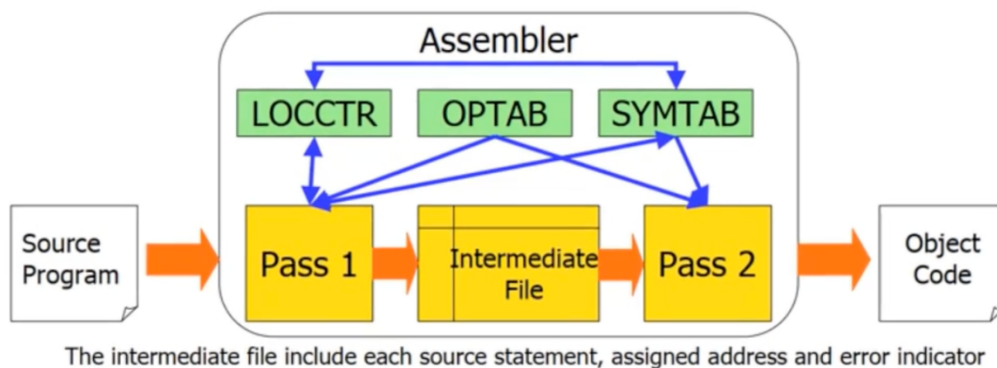
Pass 1: Find all labels and their addresses (build the symbol table).

Pass 2: Use the symbol table to generate complete object code.

A **two-pass assembler** is an assembler that makes two sequential scans over the source program:

Assembler algorithm and data structures

- OPTAB: operation code table
- SYMTAB: symbol table
- LOCCTR: location counter



Pass 1 (Define symbols):

- Assign memory addresses to all statements using a **Location Counter (LOCCTR)**.
- Build a **Symbol Table (SYMTAB)** mapping each label to its assigned address.
- Process assembler directives that affect address assignment (RESB, RESW, BYTE, WORD, EQU, ORG).
- Produce an **intermediate file** containing each source statement along with its address and error flags.
- Determine program length.

Pass 2 (Assemble):

- Read the intermediate file (or source again) with addresses already known.
- Translate mnemonic opcodes to machine code using **OPTAB**.
- Replace symbolic operands with addresses from SYMTAB.
- Generate data constants (BYTE, WORD) in binary/hex.
- Write the **object program** (Header, Text, End records) and the assembly listing.

This design cleanly handles **forward references** because all label addresses are known before code generation begins.

Why Two Passes are Needed?

Line	Label	Opcode	Operand
10	START	1000	
20		JSUB	RDREC
...			
100	RDREC	LDX	ZERO

At line 20, the assembler needs the address of `RDREC` to generate `JSUB`'s object code (e.g., `482039`). But `RDREC` is defined at line 100 – a **forward reference**.

- **One-pass assembler** would have to guess (place 0) and later backpatch – messy, especially for relocatable code.
- **Two-pass assembler:** Pass 1 records `RDREC`'s address `2039`. Pass 2 uses it directly.

Pass 1:

```
begin
  read first input line
  if OPCODE = 'START' then
    begin
      save #[OPERAND] as starting address
      initialize LOCCTR to starting address
      write line to intermediate file
      read next input line
    end {if START}
  else
    initialize LOCCTR to 0
  while OPCODE ≠ 'END' do
    begin
      if this is not a comment line then
        begin
          if there is a symbol in the LABEL field then
            begin
              search SYMTAB for LABEL
              if found then
                set error flag (duplicate symbol)
              else
                insert (LABEL,LOCCTR) into SYMTAB
              end {if symbol}
            end
          search OPTAB for OPCODE
          if found then
            add 3 {instruction length} to LOCCTR
          else if OPCODE = 'WORD' then
            add 3 to LOCCTR
          else if OPCODE = 'RESW' then
            add 3 * #[OPERAND] to LOCCTR
          else if OPCODE = 'RESB' then
            add #[OPERAND] to LOCCTR
          else if OPCODE = 'BYTE' then
            begin
              find length of constant in bytes
              add length to LOCCTR
            end {if BYTE}
          else
            set error flag (invalid operation code)
          end {if not a comment}
          write line to intermediate file
          read next input line
        end {while not END}
      write last line to intermediate file
      save (LOCCTR - starting address) as program length
    end {Pass 1}
```

Figure 2.4(a) Algorithm for Pass 1 of assembler.

Pass 2:

```
begin
  read first input line {from intermediate file}
  if OPCODE = 'START' then
    begin
      write listing line
      read next input line
    end {if START}
  write Header record to object program
  initialize first Text record
  while OPCODE ≠ 'END' do
    begin
      if this is not a comment line then
        begin
          search OPTAB for OPCODE
          if found then
            begin
              if there is a symbol in OPERAND field then
                begin
                  search SYMTAB for OPERAND
                  if found then
                    store symbol value as operand address
                  else
                    begin
                      store 0 as operand address
                      set error flag (undefined symbol)
                    end
                  end {if symbol}
                else
                  store 0 as operand address
                  assemble the object code instruction
                end {if opcode found}
              else if OPCODE = 'BYTE' or 'WORD' then
                convert constant to object code
              if object code will not fit into the current Text record then
                begin
                  write Text record to object program
                  initialize new Text record
                end
              add object code to Text record
            end {if not comment}
          write listing line
          read next input line
        end {while not END}
      write last Text record to object program
      write End record to object program
      write last listing line
    end {Pass 2}
```

Figure 2.4(b) Algorithm for Pass 2 of assembler.

Assembler Algorithm and Data Structures

Operation Code Table (OPTAB)

OPTAB (Operation Code Table) is a static data structure used by the assembler to store information about each mnemonic machine instruction. For each opcode mnemonic, OPTAB typically contains:

- **Mnemonic** (key) – e.g., LDA, ADD, JSUB
- **Machine opcode** (numeric value) – e.g., 0x00 for LDA in SIC
- **Instruction format** (1, 2, 3, or 4 in SIC/XE)
- **Instruction length** (number of bytes) – e.g., 3 for SIC standard instructions
- **Flags** – e.g., whether it uses indexed addressing, modifies condition code, etc.

OPTAB is usually **static** – its contents are fixed when the assembler is written (unlike SYMTAB, which grows during assembly). It is often implemented as a **hash table** for fast lookup.

OPTAB is used to lookup mnemonic operation codes and translate them to their machine language equivalents.

Without OPTAB, the assembler would not know how to translate mnemonics to binary – it would be useless.

During Pass 1, OPTAB is used to look up and validate operation codes in the source program.

In Pass 2, it is used to translate the operation codes to machine language.

Importance of OPTAB :

Purpose	Why needed
Opcode validation	Check if the mnemonic is valid (not a typo like LDAX).
Instruction length	In Pass 1, increment LOCCTR by the correct number of bytes.
Object code generation	In Pass 2, retrieve the numeric opcode to build the instruction.
Format selection	For SIC/XE, know whether to use Format 2 (register), Format 3 (relative), or Format 4 (extended).
Addressing mode restrictions	Some instructions cannot use certain addressing modes; OPTAB can store allowed modes.

SIC Opcode

Mnemonic	Opcode (hex)	Length (bytes)	Format
LDA	00	3	3
LDX	04	3	3
STA	0C	3	3
STX	10	3	3
ADD	18	3	3
SUB	1C	3	3
JSUB	48	3	3
RSUB	4C	3	3
COMP	28	3	3
TIX	2C	3	3
JEQ	30	3	3
JLT	38	3	3

Mnemonic	Format	Opcode	Effect	Notes
ADD m	3/4	18	$A \leftarrow (A) + (m.m+2)$	
ADDF m	3/4	58	$F \leftarrow (F) + (m.m+5)$	X F
ADDR r1,r2	2	90	$r2 \leftarrow (r2) + (r1)$	X
AND m	3/4	40	$A \leftarrow (A) \& (m.m+2)$	
CLEAR r1	2	B4	$r1 \leftarrow 0$	X
COMP m	3/4	28	$(A) : (m.m+2)$	C
COMPF m	3/4	88	$(F) : (m.m+5)$	X F C
COMPR r1,r2	2	A0	$(r1) : (r2)$	X C
DIV m	3/4	24	$A \leftarrow (A) / (m.m+2)$	
DIVF m	3/4	64	$F \leftarrow (F) / (m.m+5)$	X F
DIVR r1,r2	2	9C	$r2 \leftarrow (r2) / (r1)$	X
FIX	1	C4	$A \leftarrow (F)$ [convert to integer]	X F
FLOAT	1	C0	$F \leftarrow (A)$ [convert to floating]	X F
HIO	1	F4	Halt I/O channel number (A)	P X
J m	3/4	3C	$PC \leftarrow m$	
JEQ m	3/4	30	$PC \leftarrow m$ if CC set to =	
JGT m	3/4	34	$PC \leftarrow m$ if CC set to >	
JLT m	3/4	38	$PC \leftarrow m$ if CC set to <	
JSUB m	3/4	48	$L \leftarrow (PC); PC \leftarrow m$	
LDA m	3/4	00	$A \leftarrow (m.m+2)$	
LDB m	3/4	68	$B \leftarrow (m.m+2)$	X
LDCH m	3/4	50	A [rightmost byte] $\leftarrow (m)$	
LDF m	3/4	70	$F \leftarrow (m.m+5)$	X F
LDL m	3/4	08	$L \leftarrow (m.m+2)$	
LDS m	3/4	6C	$S \leftarrow (m.m+2)$	X
LDT m	3/4	74	$T \leftarrow (m.m+2)$	X
LDX m	3/4	04	$X \leftarrow (m.m+2)$	
LPS m	3/4	D0	Load processor status from information beginning at address m (see Section 6.2.1)	P X
MUL m	3/4	20	$A \leftarrow (A) * (m.m+2)$	

Mnemonic	Format	Opcode	Effect	Notes
MULF m	3/4	60	$F \leftarrow (F) * (m.m+5)$	X F
MULR r1,r2	2	98	$r2 \leftarrow (r2) * (r1)$	X
NORM	1	C8	$F \leftarrow (F)$ [normalized]	X F
OR m	3/4	44	$A \leftarrow (A) \vee (m.m+2)$	
RD m	3/4	D8	A [rightmost byte] $\leftarrow$ data from device specified by (m)	P
RMO r1,r2	2	AC	$r2 \leftarrow (r1)$	X
RSUB	3/4	4C	$PC \leftarrow (L)$	
SHIFTL r1,n	2	A4	$r1 \leftarrow (r1)$; left circular shift n bits. [In assembled instruction, $r2 = n-1$]	X
SHIFTR r1,n	2	A8	$r1 \leftarrow (r1)$; right shift n bits, with vacated bit positions set equal to leftmost bit of (r1). [In assembled instruction, $r2 = n-1$]	X
SIO	1	F0	Start I/O channel number (A); P X address of channel program is given by (S)	
SSK m	3/4	EC	Protection key for address m $\leftarrow (A)$ (see Section 6.2.4)	P X
STA m	3/4	0C	$m.m+2 \leftarrow (A)$	
STB m	3/4	78	$m.m+2 \leftarrow (B)$	X
STCH m	3/4	54	$m \leftarrow (A)$ [rightmost byte]	
STF m	3/4	80	$m.m+5 \leftarrow (F)$	X F
STI m	3/4	D4	Interval timer value $\leftarrow (m.m+2)$ (see Section 6.2.1)	P X
STL m	3/4	14	$m.m+2 \leftarrow (L)$	
STS m	3/4	7C	$m.m+2 \leftarrow (S)$	X
STSW m	3/4	E8	$m.m+2 \leftarrow (SW)$	P
STT m	3/4	84	$m.m+2 \leftarrow (T)$	X
STX m	3/4	10	$m.m+2 \leftarrow (X)$	
SUB m	3/4	1C	$A \leftarrow (A) - (m.m+2)$	
SUBF m	3/4	5C	$F \leftarrow (F) - (m.m+5)$	X F

Symbol Table (SYMTAB)

SYMTAB (Symbol Table) is a dynamic data structure used by the assembler to store information about each user-defined label (symbol) in the source program. For each symbol, SYMTAB typically contains:

- **Symbol name** (key) – e.g., RDREC, LOOP, FIVE
- **Value** (address or constant) – e.g., 2039 (hex)
- **Type** – absolute (constant) or relative (address within program)
- **Length** (for data areas) – optional
- **Flags** – e.g., whether the symbol is defined, referenced, external, etc.

SYMTAB is **dynamic** – it is built incrementally during Pass 1 and used in Pass 2. It is usually implemented as a **hash table** for fast insertion and lookup.

Importance of SYMTAB

Purpose	Why needed
Address resolution	Replace symbolic operands (e.g., RDREC) with numeric addresses in object code.
Forward reference handling	Record label addresses even if used before definition; Pass 2 looks them up.
Duplicate detection	Flag an error if the same label is defined twice.
Relocation information	Store whether a symbol is relative (address changes with load) or absolute (fixed).
External linking	For control sections, mark symbols as external definitions or references.
Error reporting	Detect undefined symbols at the end of Pass 1.

Symbol	Address (hex)	Type	Notes
FIRST	1000	relative	Label on instruction
CLOOP	1003	relative	Label on instruction
ENDFIL	1015	relative	Label on instruction
EOF	102A	relative	Byte constant
THREE	102D	relative	Word constant
ZERO	1030	relative	Word constant
RETADR	1033	relative	Reserved word

Location Counter (LOCCTR)

LOCCTR (Location Counter) is a variable used by the assembler during Pass 1 to keep track of the **current memory address** being assigned to each source statement. It is initialized to the starting address specified in the **START** directive (or 0 if **START** is omitted). For each statement processed, the assembler uses LOCCTR to:

- Assign an address to any label in the LABEL field.
- Determine the address of the next statement by adding the length of the current instruction or data area.

After processing the entire program, the final value of LOCCTR (minus the starting address) gives the **program length** in bytes.

LOCCTR is **not** stored in a table; it is a **runtime variable** that changes as the assembler reads each line.

LOCCTR

- Initialize to be the beginning address specified in the “START” statement
- $\text{LOCCTR} \leftarrow \text{LOCCTR} + (\text{instruction length})$
- The current value of LOCCTR gives the address to the label encountered

Importance of LOCCTR

Purpose	Why needed
Label address assignment	When a label appears, its value is the current LOCCTR.
Forward reference resolution	Labels get addresses in Pass 1; Pass 2 uses those addresses.
Program length calculation	Final LOCCTR – start address = total bytes of assembled program.
Storage reservation	For <code>RESB n</code> , LOCCTR is increased by <code>n</code> bytes.
Address consistency	Ensures each instruction or data area gets a unique, sequential address.
Object code generation	Pass 2 uses the addresses stored in the intermediate file (which came from LOCCTR in Pass 1) to compute displacements and absolute addresses.

Without LOCCTR, the assembler would have no way to know where each instruction will be placed in memory – labels would have no meaning.

Line	Source statement (simplified)	Action on LOCCTR	LOCCTR after line
5	COPY START 1000	Set LOCCTR = 1000	1000
10	FIRST STL RETADR	Add 3 (instruction length) → 1003	1003
15	CL00P JSUB RDREC	Add 3 → 1006	1006
20	LDA LENGTH	Add 3 → 1009	1009
25	COMP ZERO	Add 3 → 100C	100C
30	JBQ ENDFIL	Add 3 → 100F	100F
35	JSUB WRREC	Add 3 → 1012	1012
40	J CL00P	Add 3 → 1015	1015
45	ENDFIL LDA EOF	Add 3 → 1018	1018
50	STA BUFFER	Add 3 → 101B	101B



Machine dependent assembler features

Machine-dependent assembler features are aspects of the assembler's design and functionality that are dictated by the architecture of the target computer. They include:

- **Instruction formats** – how many bytes, layout of opcode, flags, addresses.
- **Addressing modes** – direct, indexed, immediate, indirect, PC-relative, base-relative.
- **Register set** – number, names, and special uses (e.g., base register B, index register X).
- **Instruction set** – which mnemonics exist, their opcodes, and allowed operand types.
- **Relocation** – how the assembler prepares code for loading at different addresses (using relative addressing or modification records).

These features are **not portable** – an assembler for SIC/XE cannot assemble code for a Pentium without major changes.

Instruction Formats

Format 1 (1 byte) –

8
opcode

 Example: RSUB

Format 1 (2 byte) -

8	4	4
op	r1	r2

 Example: ADDR S, T

Format 3 (3 byte) -

6	1	1	1	1	1	1	12
op	n	i	x	b	p	e	Displacement

– Example: LDA #3

Format 4 (4 byte) -

6	1	1	1	1	1	1	20
op	n	i	x	b	p	e	Address

– Example: +JSUB RDREC

Addressing Modes and Flag Bits

- Base relative (n=1, i=1, b=1, p=0)
- Program-counter relative (n=1, i=1, b=0, p=1)
- Direct (n=1, i=1, b=0, p=0)
- Immediate (n=0, i=1, x=0)
- Indirect (n=1, i=0, x=0)
- Indexing (both n & i = 0 or 1, x=1)
- Extended (e=1 for format 4, e=0 for format 3)

• Base Relative Addressing Mode

	n	i	x	b	p	e	
opcode	1	1		1	0		disp

$n=1, i=1, b=1, p=0, TA = (B) + disp \quad (0 \leq disp \leq 4095)$

• Program-Counter Relative Addressing Mode

	n	i	x	b	p	e	
opcode	1	1		0	1		disp

$n=1, i=1, b=0, p=1, TA = (PC) + disp \quad (-2048 \leq disp \leq 2047)$

- Direct Addressing Mode

	n	i	x	b	p	e	
opcode	1	1		0	0		disp

n=1, i=1, b=0, p=0, TA = disp (0 ≤ disp ≤ 4095)

	n	i	x	b	p	e	
opcode	1	1	1	0	0		disp

n=1, i=1, b=0, p=0, TA=(X)+disp (with index addressing mode)

- Immediate Addressing Mode

	n	i	x	b	p	e	
opcode	0	1	0				disp

n=0, i=1, x=0, Operand = disp

- Indirect Addressing Mode

	n	i	x	b	p	e	
opcode	1	0	0				disp

n=1, i=0, x=0, TA= (disp)

Machine Independent Assembler Features

The parts of an assembler that **do not depend on the target machine's architecture**. These features are choices made by the assembler designer to make programming easier, and they appear in similar forms across many different computers.

Machine-independent assembler features are capabilities of an assembler that are not dictated by the target machine's instruction set, addressing modes, or register organization. They include:

- **Literals** – constants written directly in the operand field (=C'EOF').
- **Symbol-defining statements** – EQU (equate) and ORG (origin).
- **Expressions** – arithmetic expressions in operands (BUFFEND-BUFFER).
- **Program blocks** – grouping parts of the program (USE directive) that are rearranged by the assembler.
- **Control sections and linking** – splitting a program into independently assembled parts (CSECT, EXTDEF, EXTREF) with external references.

These features are **portable** – an assembler for a different CPU could support the same directives with similar meanings

Why Machine Independent Features are Needed?

Feature	Why needed
Literals	Avoid defining small constants separately; make code more readable and compact.
EQU	Give symbolic names to constants and expressions; improve maintainability.
ORG	Define structured data (e.g., symbol tables) without manual address calculation.
Expressions	Compute offsets, lengths, and addresses at assembly time; reduce magic numbers.
Program blocks	Group data declarations together (e.g., all constants in one block) without forcing source order; reduces forward reference problems.
Control sections	Allow separate assembly, testing, and linking of subroutines; enable large team projects and library reuse.

These features are **independent of machine architecture** – they can be added to any assembler regardless of the underlying hardware.

Literals

A **literal** is a constant operand written in the instruction's operand field, prefixed with `=` and using the same notation as `BYTE`. The assembler:

- Places the constant's value in a **literal pool** (usually at the end of the program or at a `LTORG` directive).
- Replaces the literal in the instruction with the address where the constant is stored.
- Detects duplicate literals and stores only one copy.

Instead of Writing

```
EOF BYTE C'EOF'
```

...

```
LDA EOF
```

You write directly

```
LDA =C'EOF'
```

The assembler automatically creates a **literal pool** (a hidden data area) that contains the constant and uses its address in the instruction.

LTORG Directive

LTORG forces the assembler to place the literal pool at that point in the program. This is useful to keep literals close to instructions that use them, so that PC-relative or base-relative addressing is possible.

Symbol-Defining Statements (EQU and ORG)

EQU (Equate)

Intuition: Give a symbolic name to a value, like `#define` in C.

Syntax: `label EQU expression`

Example:

text

```
MAXLEN EQU 4096
```

```
LDT #MAXLEN ; becomes LDT #4096
```

Technical: The assembler enters `MAXLEN` into SYMTAB with value = `4096` and type = absolute. No memory is allocated.

Restriction: All symbols in the expression must be defined before this line (forward references not allowed in standard two-pass assembler; otherwise need multi-pass).

ORG (Origin)

Intuition: Change the location counter to a new value (go to a specific address).

Syntax: `ORG expression`

```
STAB RESB 1100
```

```
    ORG STAB
```

```
SYMBOL RESB 6
```

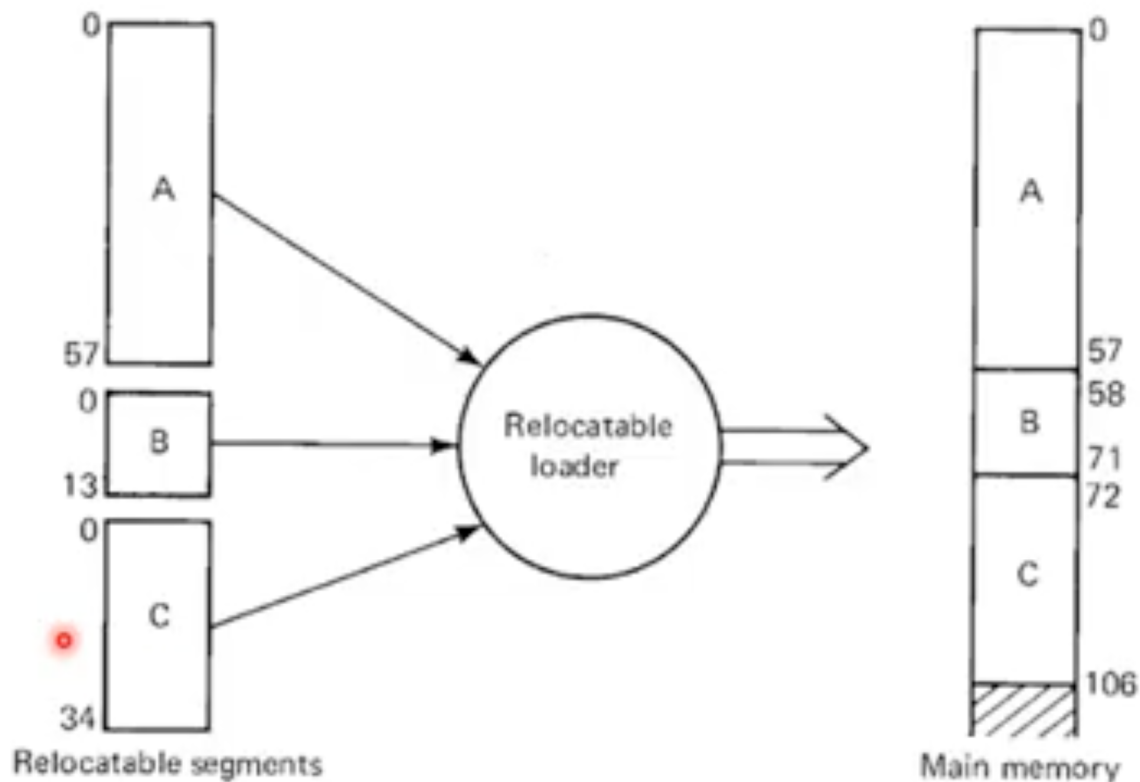
```
VALUE RESW 1
```

```
FLAGS RESB 2
```

```
    ORG STAB+1100
```

Program Relocation

Program relocation is the process of adjusting address references in a program so that it can be loaded and executed at any memory location, not just the address it was assembled for. This is essential for **multiprogramming** (running several programs concurrently) and for efficient memory management.



Absolute Program

An **absolute program** is assembled with a **fixed starting address**. All labels and addresses are assigned **absolute (real) memory addresses** by the assembler. The object code contains these absolute addresses

directly. The loader simply places the program at the specified address and transfers control. There is **no relocation information**.

Characteristics:

- Assembler directive: `START 1000` (or any nonzero address)
- Location counter starts at that address.
- All address fields in object code are **absolute**.
- Can only be loaded at the pre-defined address.
- Simple, but inflexible – cannot coexist easily with other programs.

COPY	START	1000
FIRST	LDA	ALPHA
	STA	BETA
ALPHA	WORD	5
BETA	WORD	0
	END	FIRST

The object code contains absolute addresses 1003, 1005 – these are fixed. If you try to load this program at address 2000, the instruction would still try to access 1003, which is the wrong location.

Relocatable Program

A **relocatable program** is assembled with a **relative starting address of 0** (or any base, but typically 0). All addresses are **relative offsets** from the start of the program. The assembler also generates **relocation records** that tell the loader which address fields need to be adjusted by adding the actual load address. The loader then performs the relocation at load time.

Characteristics:

- Assembler directive: **START 0**
- Location counter starts at **0**.
- Address fields in object code are **relative** (offsets).
- The object file includes **Modification records** (or relocation entries).
- Can be loaded at any memory address – the loader adds the load address to all relocatable fields.
- Required for multiprogramming, shared libraries, and position-independent code.

COPY	START	0
FIRST	LDA	ALPHA
	STA	BETA
ALPHA	WORD	5
BETA	WORD	0
	END	FIRST

Feature	Absolute Program	Relocatable Program
START directive	START 1000 (nonzero)	START 0 (zero)
Addresses in object code	Absolute (real)	Relative (offset from start)
Relocation information	None	Modification records (e.g., M00000705)
Can be loaded anywhere?	No – only at assembled address	Yes – loader adds load address
Typical use	Simple, single-tasking systems, ROM code	Multiprogramming, OS, shared libraries
Loader's job	Just copy to memory	Copy, then add load address to relocatable fields
Example object code	00102D (absolute address 102D)	4B101036 (relative address 01036)